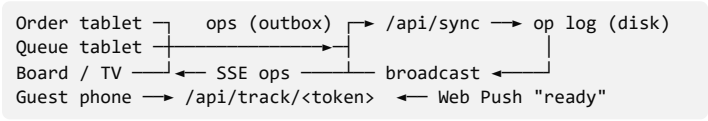
BEANS&BEATS QUEUE: TECHNICAL SUMMARY

Order & pickup system for the Beans&Beats charity coffee booth · Christmas Together 2026, Hall F, Marina Bay Sands, 26–29 Nov · runs in any browser, no app download

STACK & ARCHITECTURE

- **Server:** Node.js 22, plain `http` module, no framework, no database. One dependency (`web-push`) plus two vendored browser libraries (QR generator, jsQR reader). Deploys as one process with a persistent disk (Render Singapore blueprint or Docker).
- **Storage:** an append-only **op log** (`ops.ndjson`) replayed into memory at boot. It is crash-safe, auditable and trivially backed up. A 4-day event is around 20k ops, well under 10 MB.
- **Clients:** vanilla-JS PWA (no build step). A staff app serves Order, Queue, Ready board, Guest display, Menu and Report. A separate lightweight guest page, a service worker for the offline shell and Web Push, localStorage for the cached state and outbox.
- **Shared core:** `core.js` holds the reducer, conflict rules, report and CSV. The *same file* runs on the server and every tablet, so they can't disagree.
- **Realtime:** Server-Sent Events fan out new ops to every tablet (~100 ms), with 3 s polling as fallback. Guests poll a public endpoint every 3 s and get a Web Push on "ready".
- **Auth:** two PINs (barista, booth lead) give HMAC-signed 7-day tokens, so a tablet stays logged in through wifi loss. Lead-only ops (menu, close day, CSV) are enforced server-side.



HOW IT WORKS OFFLINE AND SYNCS

- Every action is an **op** {uuid, type, timestamp, payload}. It is written to the device's **outbox** *before* the UI updates, so a dropped connection, reload or flat battery can't lose an order. The UI renders *server state* + *pending ops*.
- The sync loop posts the outbox whenever the network allows. The server is **idempotent by op id**, so retries never duplicate, and returns everything the device missed since its cursor.
- **Conflict rules** converge regardless of arrival order. Order creation is first-write-wins on a UUID. Status and menu fields are last-writer-wins by timestamp, with the server clamping future-dated clocks. A late "making" can't undo a newer "ready".
- **Order numbers without a server:** each order station owns a number series (1 station: 001, 002…; 2 stations: odd/even). Numbers are unique offline and reset daily by Singapore date.
- The guest QR carries a snapshot of the order in the URL *fragment* (never sent to the server), so the guest page renders before the tablet has synced. The service worker caches every screen, so tablets and the guest page reload offline.
- Status pill: Live / Offline · saved on device / N to sync. ⚙ → *Simulate wifi drop* for drills. A full data reset changes a server **epoch**, and tablets then discard stale caches automatically.

BOOTH FLOW (CONFIRMED WITH BEANS&BEATS)

Guest orders at the counter → barista keys drink, hot/iced, milk/oat, first name → the POS operator keys in only the **total shown** (PayNow or card; no cash) → **Paid · PayNow / Card** issues the number and a QR on screen (tablet turned to the guest, or a guest-facing `/display`; **no paper**) → queue: one tap *Making*, one tap *Ready* → phone chimes, vibrates, turns black, push notification; the board lists the number and name → guest shows the pickup QR and the tablet/laptop **camera scans it** to mark the right order collected.

PERSONAL DATA & PDPA

DATA	WHY	RETENTION
First name	Call out / board display	Deleted at **Close day** on every device and in the on-disk log
Mobile (optional)	Reach guest about this order only; needs a ticked consent	Deleted when the order is collected or cancelled; scrubbed from disk within 10 min
Push subscription	"Ready" alert, opt-in tap	Deleted on collect / close day
Order items, time, amount, method	Sales report / POS tally	Kept (not personal)

- **Consent & purpose limitation:** mobile only with an explicit tick and stated purpose. The guest page carries a privacy notice and contact (general@beansnbeats.com).
- **Minimisation:** no surnames, emails or accounts, and **no payment data** (payments stay on the organiser's POS). CSV exports contain no names or mobiles.
- **Protection:** HTTPS only. Guest URLs use a 128-bit random token and show only that order (no lookup by number). Staff routes need a PIN token. The board shows first name + number only. Mobiles are masked in the queue.
- **Retention limitation:** automated purges as above. `reset-day.js --wipe-all` clears everything after the dry run.

DAILY REPORT

Cups, orders, total, PayNow vs card, per drink (hot/iced/oat) and per hour. The CSV has one row per order (= one POS transaction, match by time + amount), then drink lines and a summary. It is generated on the tablet, so it also works offline.

KNOWN LIMITATIONS

- If the **hall wifi is down for every device**, the order and queue tablets can't see each other's new orders until one reconnects; each keeps working on its own. **Mitigation:** a 4G travel router as the booth's own network, or take orders and run the queue on the same tablet during an outage.
- **iPhone:** push alerts need iOS 16.4+ and "Add to Home Screen". Otherwise the open page alerts with a chime and a black screen (iOS has no web vibration). Android Chrome gets full push.
- The guest's phone needs mobile data to see live updates (hall wifi isn't required).
- Single server instance (in-memory state + local disk); no horizontal scaling (not needed at ~1–2k cups/day). Back up `DATA_DIR` daily.
- Staff auth is a shared PIN per role, not per person. Device clock skew is clamped but not fully corrected.
- The POS is not integrated: the operator keys in the total by design, and the CSV is the reconciliation bridge.

Repo: README covers setup, deployment and resetting between event days · `npm test` runs an end-to-end check of sync, conflicts, CSV and PDPA wipe · All profits go to Singapore Red Cross Young Hearts.